

Battles and the High Seas (BATHS)

Technical Report

Battles and the High Seas (BATHS) project is about a game where Encounters are fought and based on rules the Encounter is either won or lost. In BATHS, there are three key personalities – ‘The Quartermaster General’, ‘The First Sea Lord’ and ‘The Admiral of the Blue’.

- **The Quartermaster General:** Responsible for maintaining the navy ships and crews. He and his department buy ships for the navy, allocate them to ports, outfit them with stores and provide them with sailors. They dispose of ships which are not seaworthy, and sell enemy ships captured during the war. They attend to all the financial aspects of running a navy.
- **The First Sea Lord:** Responsible for the strategy of running the war. She and her department maintain information about the enemy. They create a list of encounters with the enemy to be fought in the near future. Orders are dispatched by messenger to the navy’s admirals listing encounters they should undertake.
- **The Admiral of the Blue:** One of the navy’s admirals responsible for operations in the seas around France. It is his responsibility to select a squadron of ships from the reserve fleet lying in Plymouth Harbour to undertake the Sea Lord’s encounters. The Admiral is given a War Chest for Commission the Ships and fighting Encounters.

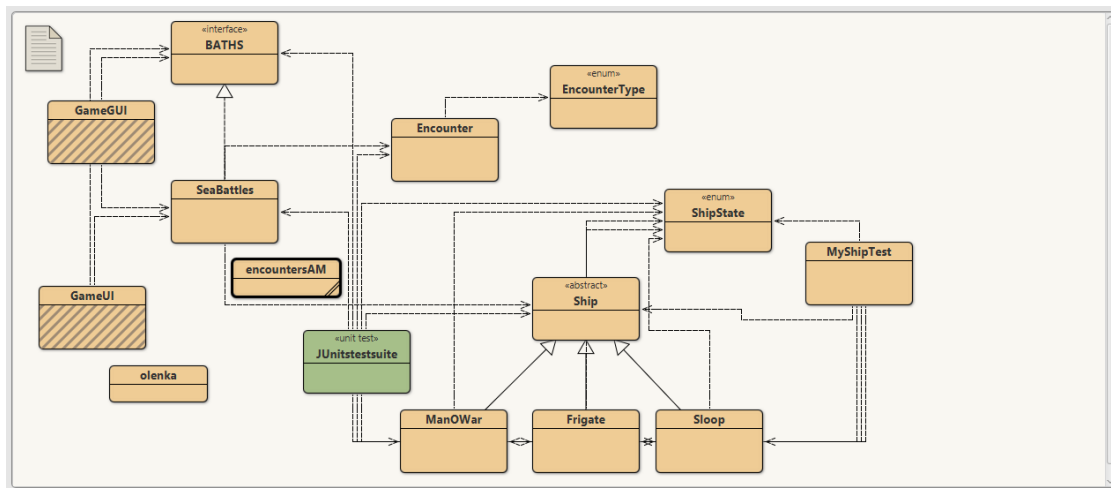
The core emphasise of this project is on the activities performed by **The Admiral of the Blue**.

The Key role of ‘The Admiral of the Blue’ are as follows:

- Admiral has a ‘War Chest’ with ‘GBP 1000’ at start of the Game
- Admiral can ‘Commission Ships’ from ‘Reserve’ to ‘Squadron’. The Admiral pays ‘Commission Fee’ from its ‘War Chest’
- Admiral can fight an Encounter by selecting an Encounter.
- If the Encounter is won, the Ship used for Encounter is put to ‘Rest’, and the Prize Money earned from the win is saved in the War Chest
- If the Encounter is lost, the Ship used for Encounter gets ‘Sunked’, and the Prize Money is deducted from the War Chest
- Admiral can De-Commission Ships from Squadron (ie Ships with Status as Active or Rested) to the Reserve. This enabled the Admiral to get 50% of the ship Commission Fee into its War Chest. (Sunken Ships cannot be De-Commisioned)
- Admiral can store ‘Rested Ship’ as ‘Active Ship’ and use them for encounters. (Sunken Ships cannot be Restored)
- At any given time if the War Chest is less than Zero and there are no Ship in Squadron (ie Ships with Status as Active or Rested), the Admiral loses the game, and also his Job

The game is being developed using object-oriented Java. The core focus of this report is to present a UML diagram representing the project's architecture consisting of the Classes its relationship and utilization. Highlight the three significant design decisions that guided our development process. The emphasis is on Decision, Alternate Solutions, Pro and Cons and decision to justify our final implementation strategy.

UML Diagram (BlueJ style) of our Project design showing relationships between Classes



List of Classes are as follows:

- Ship (Abstract Parent-Class) and ManOWar, Frigate and Sloop as its Child-Classes
 - ShipState, a enum class for State of the Ships
 - MyShipTest for basic testing of different kind of ships
- Encounter containing fields and methods for Encounter
 - EncounterTest, a enum class for Type of Encounters
- BATHS as Interface and SeaBattles as main Class.
 - encountersAM, a text file containing list of Encounters with comma separated values
- GameUI a text based menu option for executing various methods/ functions of the game
 - Olenka, a data file which saves the game and also loads the game from the same file
- GameGUI a GUI based menu option for executing various methods/ functions of the game
- JunitTestSuite (Contains multiple Junit Test Suite as listed below)
 - T1InitializationTest
 - T2EncountersTest
 - T3ShipsTest
 - T4ShipCommissionedTest
 - T5FightEncounterTest
 - T6ShipDecommissionTest
 - T7ChallengeSequences

Design and Development

During the 'Design and Development' period many points were discussed. The three main points of discussion wrt Alternate Implementations, their Pros and Cons and the Final Decision taken are being elaborated below:

1. Design Decision: Use of Inheritance for Ship related Information.

About Inheritance: In Java, Inheritance is one of the core concept of object-oriented programming (OOP) that allows one Class (called the Sub-Class or Child-Class) to inherit Fields and Methods from another class (called the Super-Class or Parent-Class). Inheritance promotes reusability of code, polymorphism, and a clean, modular structure. Where required the additional Fields can be added to the Sub-Class or Child-Class, and as required Methods of Super-Class or Parent-Class can be Overridden in the Sub-Class or Child-Class.

The requirement of having different types of Ships (viz Frigate, ManOWar, and Sloop) with some common information such as Name of the Ship, Captain, Battle Skill, Commission Fee, et al, and some specific features like Number of Decks, Number of Cannons, Number of Marines, existence of Doctors and Pinnacle subject to specific type of Ship (viz Frigate, ManOWar, and Sloop). Possibility of adding more type of ship to the Game would also be taken into design consideration. It was decided to use concept of Inheritance for Ship related Information. With 'Ship' as an 'Abstract Parent-Class', and having multiple 'Child-classes' (viz Frigate, ManOWar, and Sloop). This also allow possibility of adding more type of Ships with minimum changes.

Alternate Implementations:

Having separate classes with each Class having complete set of Fields and Methods as per ship's specification. In other words have three Classes Frigate, ManOWar, and Sloop, with their Fields and required Methods. As and when required new Classes can be added for new type of Ships.

Pros and Cons:

The four clear advantages of using Inheritance are as follows:

- **Code Reusability:** Instead of rewriting the same code in different Classes, you can define common Fields and Methods in the Parent-Class and let Child-classes inherit it.
- **Polymorphism:** Inheritance enables polymorphism, where a parent class reference can point to a child class object. This allows you to write more flexible and scalable code.
- **Maintainability and Extensibility:** Maintaining Code is easier in Inheritance as code are at under common locations. Due to Inheritance, you can create specialized Child-Classes without changing the existing Parent-Class.
- **Implementing Abstract Classes and Interfaces:** Inheritance is also used when working with abstract classes and interfaces to enforce structure in large applications

Alternate Implementations of having separate classes would only create more Maintaining Code due to similar code repeated across multiple classes. In case of Extensibility, new classes to be added would create upstream changes in the system which is not a right approach.

Team Final Decision:

It was decided to use concept of Inheritance for Ship related Information. With 'Ship' as an 'Abstract Parent-Class', and having multiple 'Child-classes' (viz Frigate, ManOWar, and Sloop). This also allow possibility of adding more type of Ships with minimum changes.

2. Design Decision: Class SeaBattles to be Central of Game Logic

The SeaBattles Class with BATHS as Interface was made available. It was envisaged by design that this class to be core of the logic for game and use other classes for respective functionalities.

Alternate Implementations:

Possibility was there to add on more classes for managing certain parts of job which can be done at SeaBattles Class level. Certain jobs could be delegated to some other classes like a FleetManager which could perform some task related to 'Ships' or EncounterManager to perform some task related to 'Encounter'. These classes to act as feeder to the core class SeaBattles.

Pros and Cons:

- Centralizing logic and having all functionality in one place makes the code easier to maintain and debug. It also makes it straightforward to test game features using unit tests, as we can initialize a SeaBattles object and invoke methods directly.
- Distributing responsibilities across multiple classes would have improved modularity, future scalability and testability. However, it would also add complexity to a small project of this size, with more class interactions to manage, upgrade and test.

Team Final Decision:

It was decided to embed the core required logic into SeaBattles, which in turn would use the Ship Parent-Classes and Child-Classes for managing ship related functionalities and Encounter for managing encounter required functionalities.

3. Design Decision: Use of ArrayList

Class SeaBattles require to store data related to all Ships (of all Types) and Encounters. This data would need to be searched, retrieved, and updated, from different functionalities/ methods. Keeping in view the quantum of data and access of such information in SeaBattles, storing data in ArrayList was preferred.

Alternate Implementations:

Alternate to ArrayList would be storing data in HashMap. It Stores key-value pairs, allows fast lookups by key. In HashMap, the Keys are unique, values can be duplicates. Unordered by default

Pros and Cons:

- For handling data for such a small size project use of 'ArrayList' would be sufficient.
- Also, When you just need a list of values (no need for keys), ArrayList is much simpler and cleaner.
- ArrayList allows O(1) access by index, like an array. HashMap doesn't have any concept of ordering or indexing elements this way.
- If you need to store duplicate items, ArrayList allows this. HashMap requires unique keys.
- Iterating over a simple list with a loop is more straightforward with an ArrayList.
- HashMap consumes slightly higher memory usage due to hashing overhead and sometimes can have hash collisions.

Team Final Decision:

Keeping in view the volume of data and use of accessibility of data related to all Ships and all Encounters it was decided to use ArrayList for storing data related to all Ships and all Encounters.